Izveštaj o testiranju performansi sistema (Load Testing)

Projekat: Smart Manufacturing

Kandidat: Milan Lazarevic (Student 1)

Alat za testiranje: Locust

1. Uvod i metodologija testiranja

U okviru verifikacije nefunkcionalnih zahteva sistema, sprovedeno je testiranje performansi (Load i Stress testing) korišćenjem alata Locust. Cilj testiranja je ispitivanje ponašanja aplikacije pri različitim nivoima opterećenja, u rasponu od 10 do 10.000 istovremenih korisnika.

Za testiranje je odabrano 10 najkritičnijih endpoint-ova, fokusirajući se na one koji su vremenski najzahtevniji (kompleksni upiti, rad sa fajlovima, analitika) i one koji trpe najveći broj konkurentnih zahteva. Metodologija obuhvata postepeno povećanje broja korisnika kako bi se identifikovala uska grla (bottlenecks) i tačka pucanja (breaking point) sistema.

Specifikacija testnog okruženja:

- **Računar:** Macbook Air M1 (Apple M1 čip, 8GB RAM)
- **OS:** macOS Tahoe
- **Napomena:** Testiranje je vršeno u lokalnom okruženju gde su baza podataka, backend aplikacija i alat za generisanje opterećenja pokrenuti na istoj mašini.

2. Analiza performansi izabranih endpoint-ova

2.1. Pretraga menadžera: *GET /managers/search*

Opis i cilj testa:

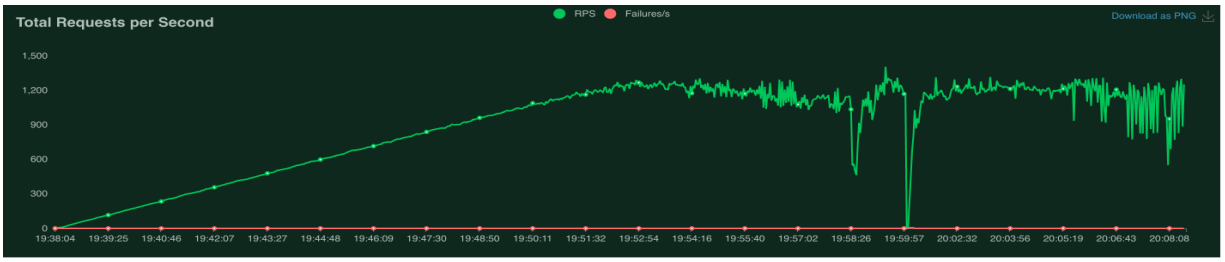
Ovaj endpoint simulira čestu operaciju pretrage korisnika. Testira se *GET* zahtev sa promenljivim query parametrima za pretragu (npr. q=a, q=m, q=986). Uz svaki zahtev vrši se provera JWT tokena (autentifikacija), pri čemu svaki simulirani korisnik prvo prolazi kroz proces prijave (*POST /api/auth/login*). Cilj je proveriti efikasnost čitanja iz baze sa filtriranjem.

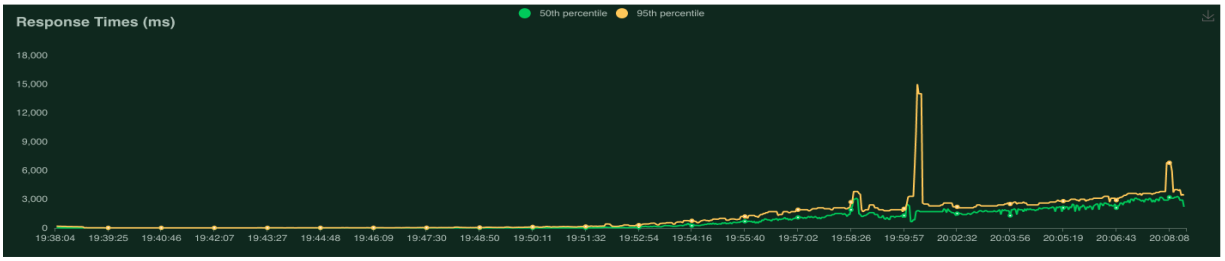
Tabelarni prikaz rezultata:

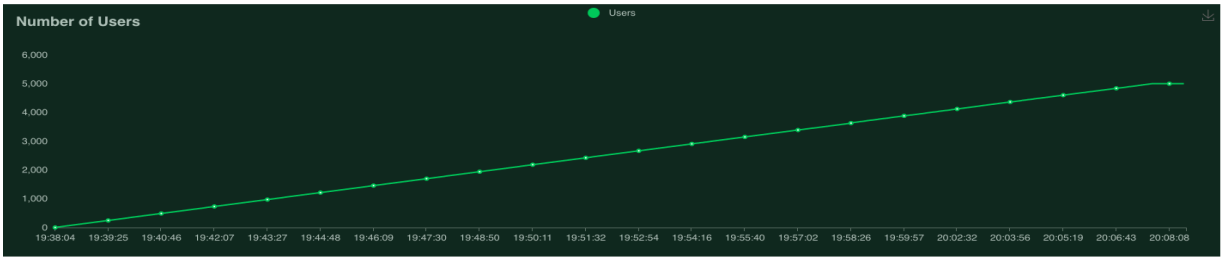
Broj korisnika	RPS	Medijana (ms)	90%ile (ms)	Max vreme (ms)	Greške (%)
10	*4.9*	*19*	*23*	*55*	*0*
100	*49.7*	*11*	*17*	*47*	*0*
500	*249.2*	*7*	*10*	*164*	*0*
1000	*499.8*	*6*	*10*	*1.435*	*0*
5000	*1329.2*	*59*	*220*	*15.685*	*0*
10000	*456*	*3800*	*8000*	*71.187*	*1.13*

Grafički prikaz rezultata:

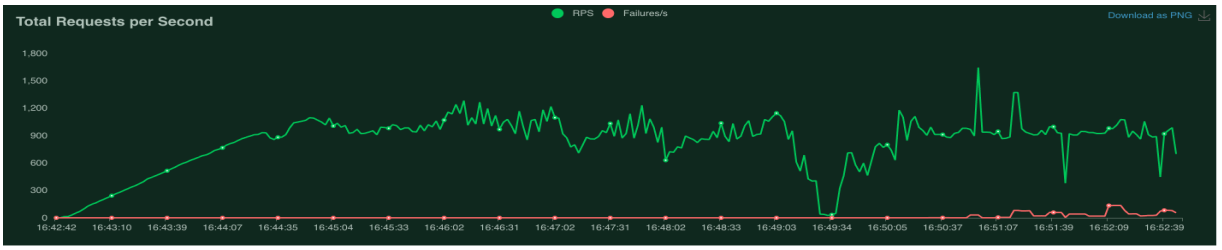
- **5000 korisnika:**

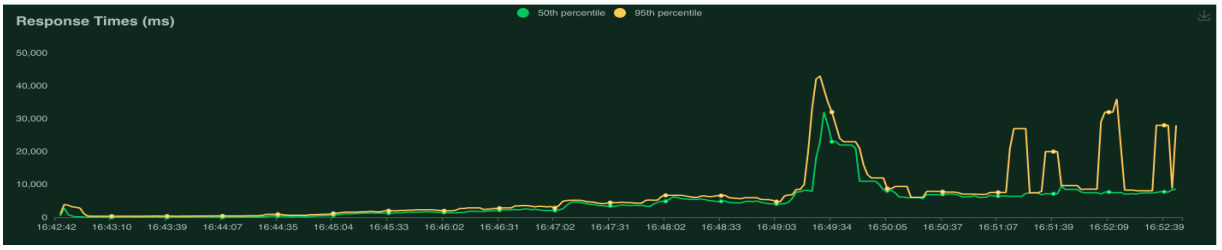


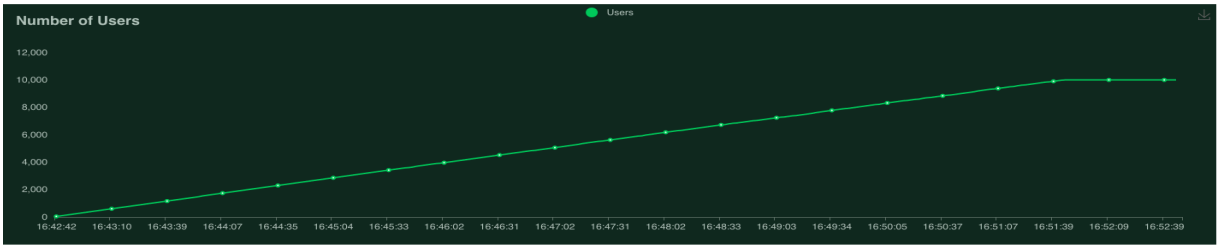




- **10000 korisnika:**







Zapažanja i analiza:

1. **Inicijalna stabilnost i "Warm-up" efekat:** Pri opterećenju od 10 do 1.000 korisnika, sistem pokazuje izuzetnu stabilnost. Zanimljivo je da vreme odziva (medijana) zapravo *opada* sa rastom korisnika (sa 19 ms pri 10 korisnika, na impresivnih 6 ms pri 1.000 korisnika). Ovo jasno ukazuje na uspešno "zagrevanje" sistema (JVM warm-up), efikasnu inicijalizaciju konekcionog pula (Connection Pool) za bazu podataka i aktivaciju mehanizama keširanja.
2. **Degradacija pod visokim opterećenjem:** Na 5.000 korisnika sistem uspeva da obradi skoro 1.330 zahteva u sekundi (RPS) bez ijedne greške. Međutim, maksimalno vreme odziva skače na preko 15 sekundi za pojedinačne zahteve, što ukazuje na to da redovi za čekanje (queues) na serveru počinju da se pune.
3. **Tačka pucanja (Breaking point):** Pri skoku na 10.000 korisnika, propusna moć drastično pada (sa 1.329 na 456 RPS), a sistem ulazi u stanje zagušenja. Javljaju se prve greške (oko 4259 neuspelih zahteva za pretragu). Analizom logova utvrđeno je da greške nisu posledica loših upita u bazi, već iscrpljivanja HTTP worker thread-ova na serveru (ConnectTimeoutError) i lančane reakcije izazvane otkazivanjem */auth/login* endpoint-a koji je neophodan za JWT validaciju.

Predlozi za unapređenje:

Pošto su indeksi i keširanje baze već implementirani i pokazuju odlične rezultate do 5.000 korisnika, glavni predlog je arhitektonske prirode: ograničiti maksimalni broj istovremenih konekcija ka bazi i implementirati *Rate Limiting* (ograničavanje broja zahteva po klijentu) kako sistem ne bi pokušavao da obradi saobraćaj koji prevazilazi njegove hardverske kapacitete, već bi graciozno odbijao višak zahteva. Takođe, razmotriti asinhronu obradu ili skaliranje autentifikacionog servisa.

2.2. Pretraga, filtriranje i paginacija proizvoda: *GET /products*

Opis i cilj testa:

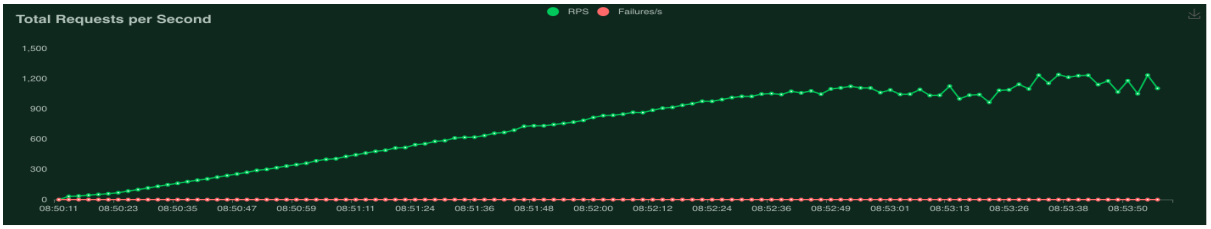
Ovaj endpoint predstavlja jedan od najopterećenijih delova sistema jer omogućava listanje kataloga dostupnih proizvoda. Testira se GET zahtev sa parametrima za dinamičku paginaciju (stranice 0-5, veličina 20), sortiranje po nazivu i filtriranje (available=true). Na backend strani, implementacija se oslanja na Spring @Cacheable apstrakciju na servisnom sloju i nativne PostgreSQL upite koji podržavaju Full-Text Search. Cilj testa je ispitati propusnu moć aplikativnog servera pri velikom broju konkurentnih čitanja kada je odgovor pretežno keširan, kao i pronalaženje tačke pucanja (breaking point) sistema.

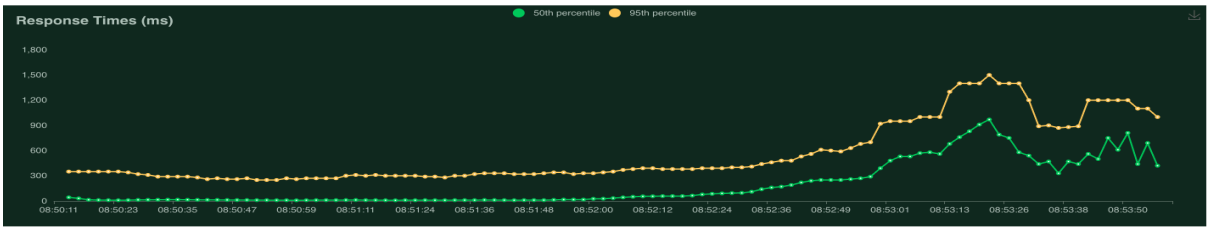
Tabelarni prikaz rezultata:

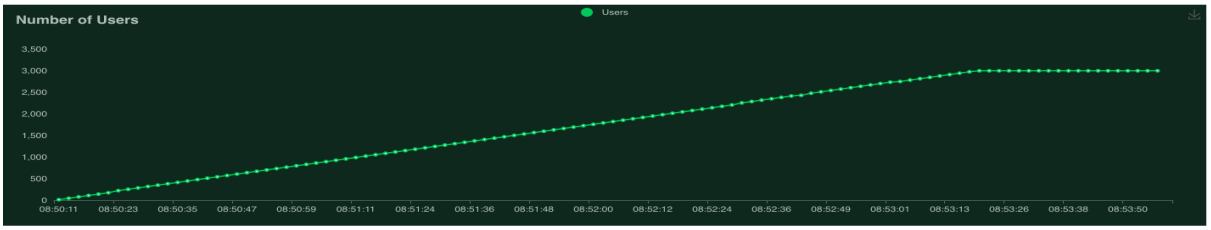
Broj korisnika	**RPS**	**Medijana (ms)**	**90%ile (ms)**	**Max vreme (ms)**	**Greške (%)**
10	*5.4*	*11*	*22*	*253*	*0*
100	*50.1*	*12*	*20*	*63*	*0*
500	*248.5*	*8*	*16*	*89*	*0*
1000	*498.9*	*7*	*57*	*281*	*0*
3000	*1176.9*	*190*	*860*	*2182*	*0*
5000	*809.6*	*1500*	*3500*	*6668*	*0*
9000	*830.8*	*3900*	*8900*	*75644*	*1*

Grafički prikaz rezultata:

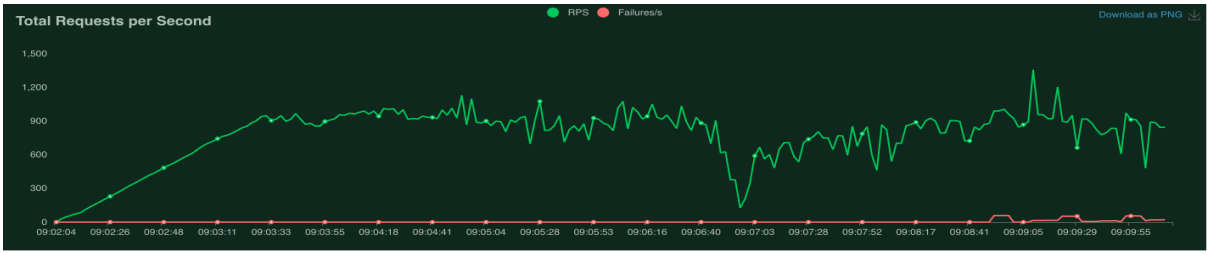
- **3000 korisnika:**

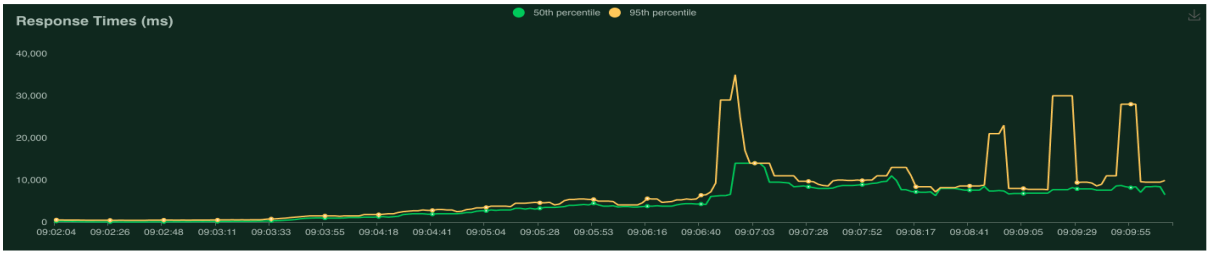


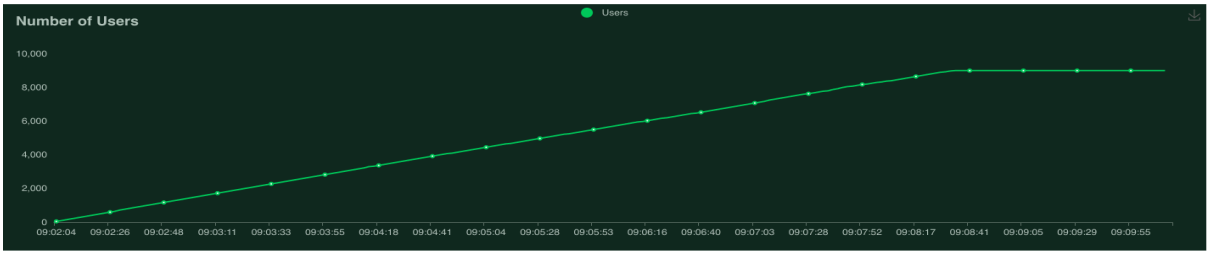




- **8000 korisnika:**







Zapažanja i inženjerska analiza uskih grla:

- **Izuzetna efikasnost keširanja i "Sweet spot":** Pri opterećenju do 1.000 korisnika, sistem ostvaruje fantastične rezultate sa medijanom odziva od samo 7 ms. Sistem dostiže svoj vrhunac propusne moći na 3.000 konkurentnih korisnika, gde uspešno obrađuje impresivnih **1.176 zahteva u sekundi (RPS)** bez ijedne greške. Ovo potvrđuje da je keširanje na servisnom sloju implementirano optimalno.
- **Degradacija usled iscrpljivanja HTTP niti (Thread Contention):** Prilikom skoka na 5.000 korisnika, RPS opada (na oko 809), a vreme odziva raste (medijana 1,5 sekundi). Iako se podaci vraćaju iz keša, sam web server (Tomcat) troši previše resursa na promenu konteksta između niti (*Context Switching*) jer pokušava da održi preveliki broj istovremenih konekcija.
- **Tačka pucanja (Breaking point) na 9.000 korisnika:** Sistem konačno popušta pod ekstremnim opterećenjem od 9.000 korisnika, pri čemu se javlja 1% grešaka, a maksimalno vreme čekanja dostiže preko 75 sekundi.
- **Analiza lančanih grešaka (Cascading failures):** Analizom *Error log-a* utvrđeno je da greške prate klasičan obrazac lančanog otkazivanja usled zagušenja mreže i servera:
    1. **ConnectTimeoutError (1299 pojava):** Web server više nije u stanju da prihvati nove konekcije na vreme jer su redovi čekanja (connection queues) operativnog sistema prepunjeni.
    2. **Otkazivanje autentifikacije (703 pojave):** Zbog opšteg zagušenja servera, endpoint */api/auth/login* počinje da otkazuje, pa simulirani klijenti ne uspevaju da dobiju JWT token.
    3. **HTTP 403 Forbidden (386 pojava):** Kao direktna posledica pada login servisa, klijenti pokušavaju da pristupe zaštićenom */products* endpoint-u bez validnog tokena, što rezultuje statusom 403.

Predlozi za unapređenje:

S obzirom na to da je trenutna implementacija na nivou koda visoko optimizovana (keširanje daje maksimalne rezultate do 3.000 korisnika), uzroci pada pri ekstremnim opterećenjima leže u ograničenjima hardvera i mrežnog steka jedne instance servera.

Za postizanje stabilnosti na preko 5.000 istovremenih korisnika, preporučuju se sledeći arhitektonski koraci:

- **Uvođenje Rate Limiting mehanizma:** Aplikacija bi trebala da na nivou API Gateway-a odbaci višak zahteva koji prevazilaze njene kapacitete (vraćanjem statusa *429 Too Many Requests*), čime bi se sprečilo potpuno zagušenje servera i omogućilo stabilno opsluživanje postojećih korisnika.
- **Razdvajanje servisa (Microservices / Izolacija):** Otkazivanje *auth* servisa usled zagušenja *products* zahtevima ukazuje na potrebu za razdvajanjem procesa autentifikacije od procesa posluživanja resursa.
- **Horizontalno skaliranje:** Za podnošenje opterećenja od blizu 10.000 korisnika, neophodno je dodati *Load Balancer* i višestruke instance aplikacije (ili preći na kontejnerizovano orkestriranje, npr. Kubernetes) kako bi se saobraćaj i mrežne konekcije ravnomerno raspodelile.

2.3. Dohvatanje entiteta po identifikatoru: GET /products/{id}

Opis i cilj testa:

Ovaj endpoint simulira operaciju preuzimanja detalja o specifičnom zapisu na osnovu njegovog jedinstvenog identifikatora (ID). Ovo je jedna od najčešćih operacija u sistemu, a pretraga se u bazi oslanja na primarni ključ, što bi po definiciji trebalo da predstavlja najbrži mogući upit.

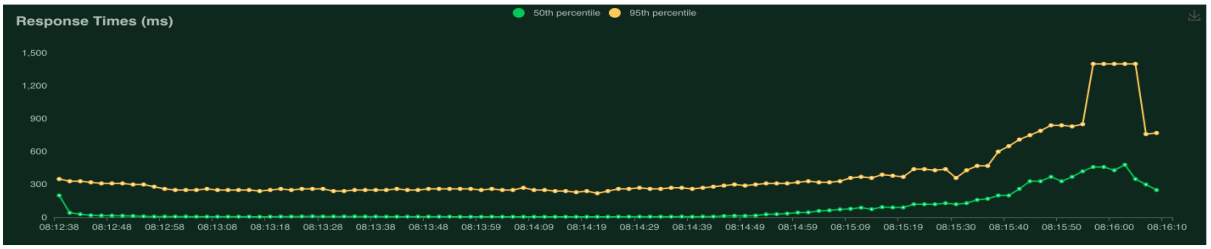
Cilj testa je ispitati propusnu moć sistema (kako web servera, tako i konekcija ka bazi) prilikom ogromnog broja jednostavnih, ali izuzetno čestih konkurentnih zahteva.

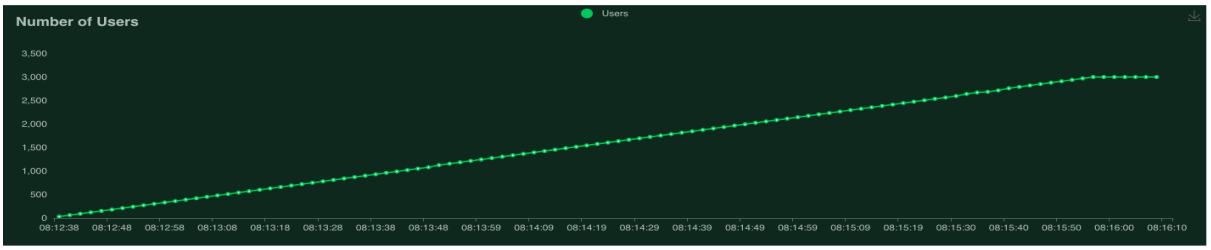
Tabelarni prikaz rezultata:

Broj korisnika	**RPS**	**Medijana (ms)**	**90%ile (ms)**	**Max vreme (ms)**	**Greške (%)**
10	*4.7*	*41*	*52*	*358*	*0*
100	*49.9*	*11*	*25*	*63*	*0*
500	*249.7*	*8*	*162*	*173*	*0*
1000	*499.8*	*6*	*48*	*261*	*0*
3000	*1314*	*75*	*470*	*2262*	*0*
5000	*982.8*	*670*	*2800*	*4168*	*0*
8000	*890.4*	*2800*	*5800*	*8723*	*0*

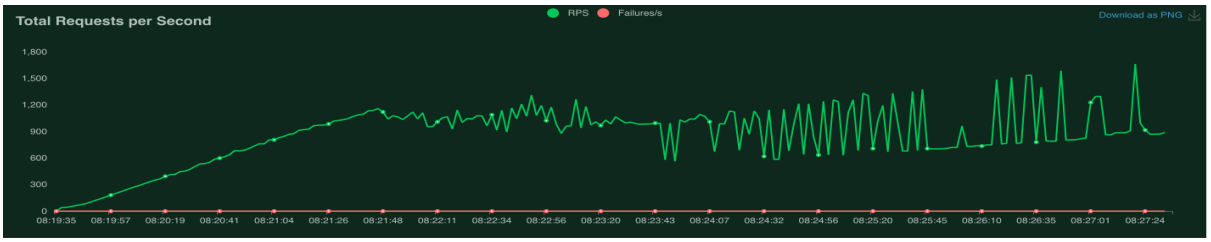
Grafički prikaz rezultata:

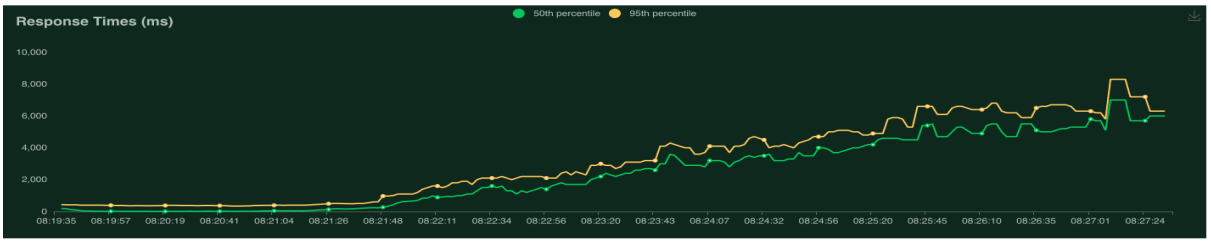
- **3000 korisnika:**

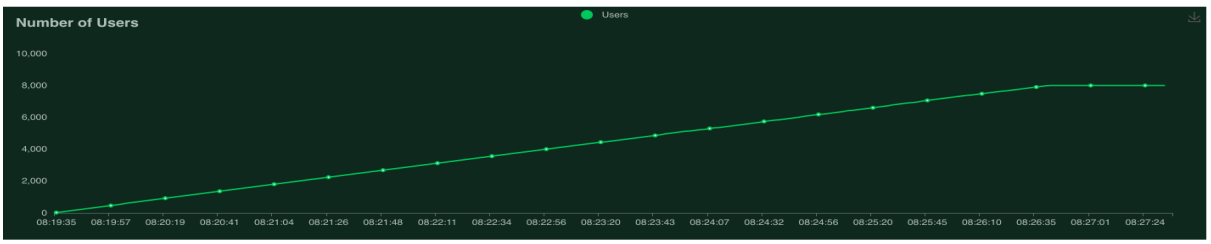




- **8000 korisnika:**







Zapažanja i inženjerska analiza uskih grla:

- **Visoka efikasnost pri umerenom opterećenju:** Kao i u prethodnim testovima, sistem pokazuje odličan proces "zagrevanja". Vreme odziva dramatično opada sa 41 ms pri 10 korisnika na svega 6 ms pri 1.000 korisnika. Baza podataka i ORM sloj besprekorno rukuju upitima po primarnom ključu u ovom opsegu.
- **Tačka maksimalne propusnosti (Sweet spot):** Pri opterećenju od 3.000 korisnika, sistem dostiže svoj apsolutni vrhunac u propusnoj moći, uspešno obrađujući impresivnih **1.314 zahteva u sekundi (RPS)**, uz sasvim prihvatljivo medijalno vreme odziva od 75 ms i bez ijedne greške.
- **Zagušenje i oscilacije (Thread Contention):** Prilikom povećanja opterećenja na 5.000 i 8.000 korisnika, uočava se jasan pad RPS-a (sa 1.314 na 890) i drastičan skok latencije (medijana raste na 2,8 sekundi). Grafikon RPS-a za 8.000 korisnika pokazuje izrazito "testerast" (oscilirajući) oblik. Ovakvo ponašanje je klasičan indikator da server troši previše resursa na promenu konteksta između niti (Context Switching) i da se aktivira intenzivno čišćenje memorije (Garbage Collection). Web server se bori da opsluži sve aktivne konekcije, stavljajući ih u red čekanja na nivou OS-a, ali ono što je izuzetno pohvalno jeste da **sistem i dalje ne puca (0% grešaka)**, već samo usporava.

Predlozi za unapređenje:

S obzirom na to da su upiti po primarnom ključu na nivou baze podataka po prirodi optimalni i pretraga preko indeksa funkcioniše besprekorno, glavna uska grla pri ekstremnim opterećenjima postaju ograničenja samog hardvera i mrežnog protokola jedne instance aplikacije.

Za sisteme koji ciljaju na kontinuirano opsluživanje preko 5.000 istovremenih korisnika, preporučuju se sledeća arhitektonska unapređenja:

- **Rate Limiting / API Gateway:** Implementacija mehanizama za ograničavanje broja upita po klijentu sprečila bi nagla zagušenja i omogućila sistemu da radi na svojoj optimalnoj granici (oko 3.000 korisnika) bez degradacije performansi za sve korisnike.
- **Horizontalno skaliranje i Load Balancing:** Trenutni rezultati na jednoj instanci (Macbook Air M1) su izuzetni, ali za dalje skaliranje neophodno je dodati Load Balancer i podići više instanci aplikativnog servera. Time bi se dolazni saobraćaj ravnomerno distribuirao, a problem iscrpljivanja Thread Pool-a jedne instance bi bio rešen.

2.4. Napredna pretraga i filtriranje fabrika: GET /filter

Opis i cilj testa:

Ovaj endpoint omogućava napredno pretraživanje i filtriranje entiteta fabrika na osnovu višestrukih opcionih parametara (lokacija, status, tekstualna pretraga), uz podršku za paginaciju i sortiranje. Na backend strani, implementirano je aplikativno keširanje, ali se usled prirode operacije podaci često preuzimaju iz baze putem složenih nativnih PostgreSQL upita koji uključuju *Full-Text Search* (tsvector).

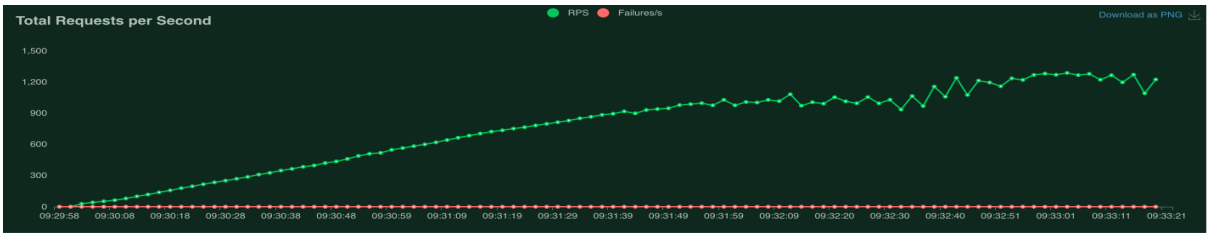
Cilj testa je ispitati ponašanje sistema pod visokim opterećenjem kada dolazi do velike varijacije u parametrima pretrage (visoka kardinalnost upita), što umanjuje efikasnost keširanja i stavlja fokus na procesorsku moć baze podataka i izvršavanje pratećih COUNT operacija.

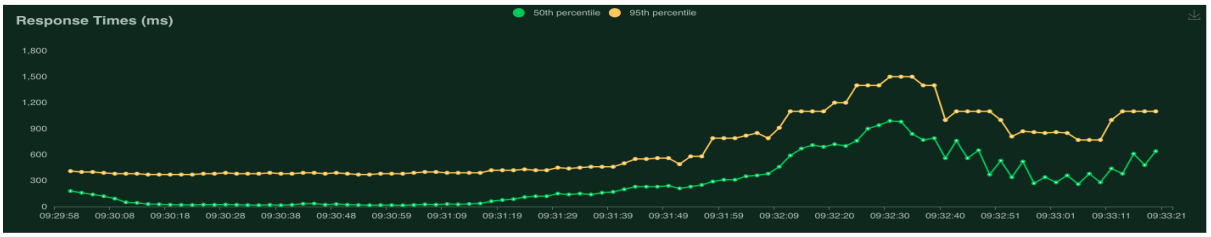
Tabelarni prikaz rezultata:

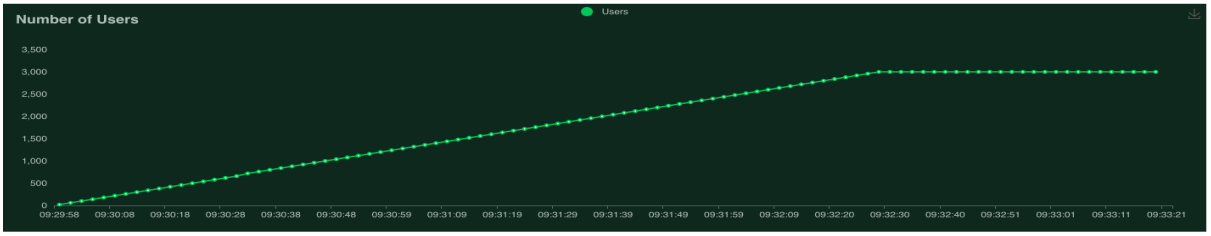
Broj korisnika	**RPS**	**Medijana (ms)**	**90%ile (ms)**	**Max vreme (ms)**	**Greške (%)**
10	*5.1*	*12*	*23*	*87*	*0*
100	*50.8*	*13*	*20*	*60*	*0*
500	*247.9*	*9*	*17*	*159*	*0*
1000	*504.7*	*7*	*76*	*363*	*0*
3000	*1177.7*	*300*	*940*	*2054*	*0*
5000	*1063.4*	*1000*	*2800*	*5507*	*0*
8900	*751.2*	*5900*	*8100*	*106984*	*1*

Grafički prikaz rezultata:

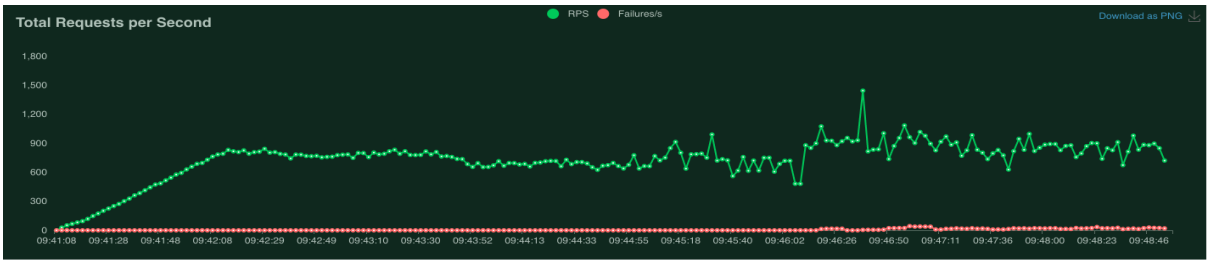
- **3000 korisnika:**

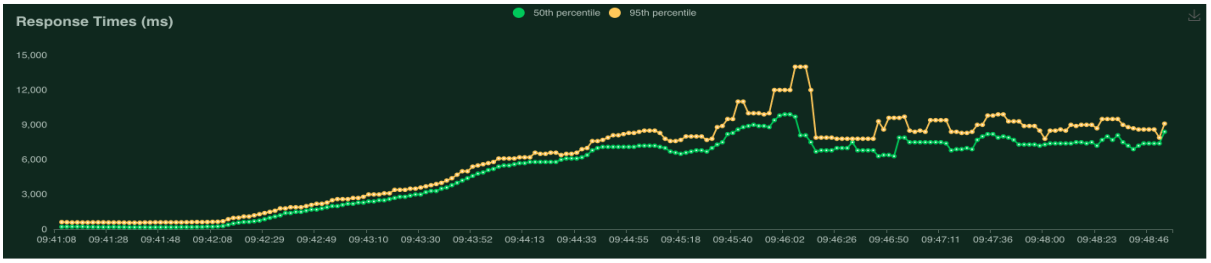


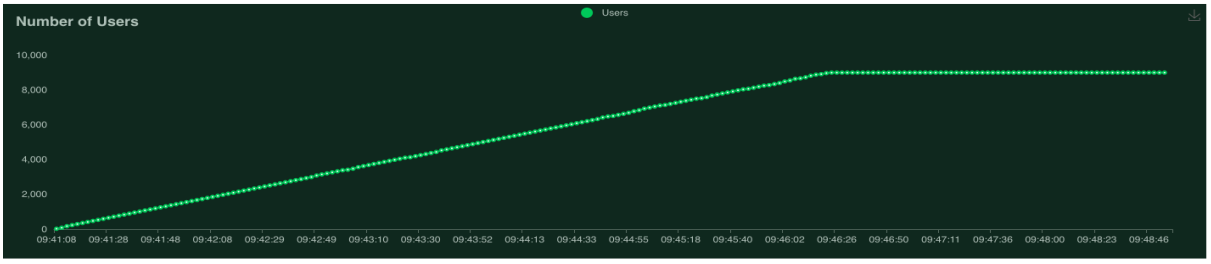




- **8900 korisnika:**







Zapažanja i inženjerska analiza uskih grla:

- **Stabilnost pri umerenom opterećenju:** Sistem pokazuje odlične performanse do nivoa od 1.000 korisnika. Vreme odziva ostaje izuzetno nisko (medijana 7 ms), što potvrđuje da kombinacija aplikativnog keširanja za najčešće upite i internog keša baze podataka (Buffer Cache) funkcioniše efikasno.
- **Maksimalna propusna moć i uticaj kardinalnosti:** Sistem dostiže svoju maksimalnu propusnu moć na opterećenju od oko 5.000 korisnika, obrađujući stabilnih **1.063 zahteva u sekundi**. Međutim, uočava se jasan rast latencije (medijana skače na 1 sekundu). Usled velikog broja različitih kombinacija filtera, aplikativni keš se zaobilazi, pa baza podataka preuzima teret izvršavanja kompleksnih upita i kalkulacije ukupnog broja rezultata (COUNT).
- **Tačka pucanja (Connection Pool Exhaustion):** Pri ekstremnom skoku na 8.900 korisnika, arhitektura popušta. RPS opada na 721, a latencija probija granice prihvatljivog (maksimalno vreme čekanja raste na preko 116 sekundi), uz pojavu 1% grešaka. Baza podataka više nije u stanju da dovoljno brzo opsluži sve kompleksne upite, što dovodi do zadržavanja konekcija. Krajnji rezultat je iscrpljivanje *Connection Pool*-a (HikariCP) i otkazivanje zahteva na nivou web servera zbog isteka vremena (timeout).

Predlozi za unapređenje:

S obzirom na to da je standardno keširanje nedovoljno efikasno za endpoint-ove sa ogromnim brojem kombinacija pretrage, arhitektonska unapređenja bi se trebala fokusirati na optimizaciju rada sa bazom podataka:

- **Dinamičko građenje upita (Optimizacija plana izvršavanja):** Trenutna upotreba nativnih upita sa "catch-all" logikom (:param IS NULL OR ...) sprečava PostgreSQL *Query Planner* da izabere optimalne indekse, jer mora da kreira generički plan izvršavanja. Prelazak na dinamičko građenje upita (npr. *JPA Criteria API* ili *QueryDSL*), gde se upitu dodaju samo oni uslovi koji su zaista prosleđeni, značajno bi ubrzao rad baze.
- **Optimizacija PostgreSQL Count upita:** Prilikom pretrage ogromne količine podataka, COUNT operacija postaje usko grlo. Preporučuje se prelazak sa Page na Slice interfejs tamo gde tačan broj stranica nije striktno neophodan, čime se izbegava jedno celo, veoma skupo skeniranje tabele.
- **Razdvajanje upisa i čitanja (Read Replicas):** Za sisteme sa izrazito velikim brojem kompleksnih operacija čitanja (Read-Heavy), dugoročno rešenje predstavlja konfigurisanje *Read Replica* instanci baze podataka, čime bi se glavni server rasteretio i omogućilo horizontalno skaliranje upita za filtriranje.

2.5. Analitika proizvodnje: *GET /factories/{id}/production-analytics*

Opis i cilj testa:

Ovaj endpoint simulira složene analitičke operacije nad vremenskim serijama (Time-Series data). Poslovna logika zahteva preuzimanje metapodataka o fabrici iz relacione baze (PostgreSQL) i uparivanje sa agregiranim produkcionim podacima koji se povlače iz InfluxDB-a korišćenjem *Flux* upitnog jezika.

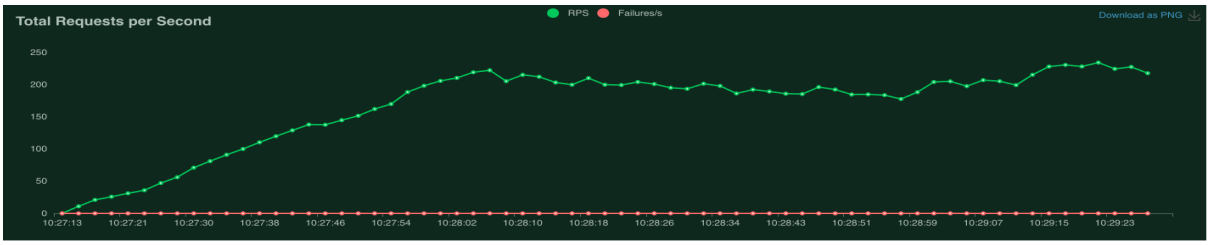
Iako je na servisnom sloju implementirana @Cacheable apstrakcija, cilj testa je ispitati ponašanje sistema i integraciju sa eksternom Time-Series bazom pod visokim opterećenjem pri generisanju dinamičkih izveštaja (za poslednja 24 sata, nedelju dana ili mesec dana).

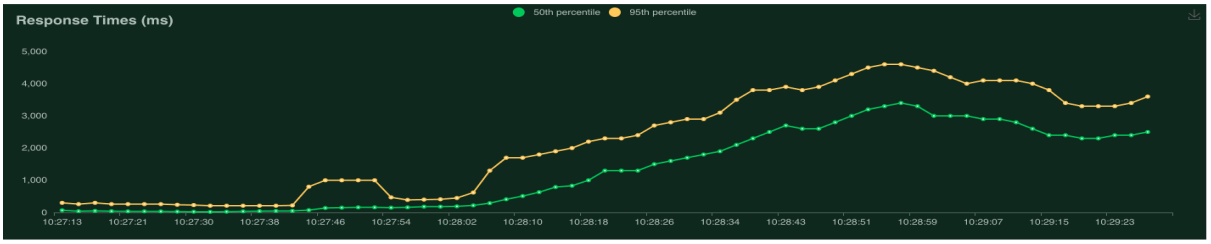
Tabelarni prikaz rezultata:

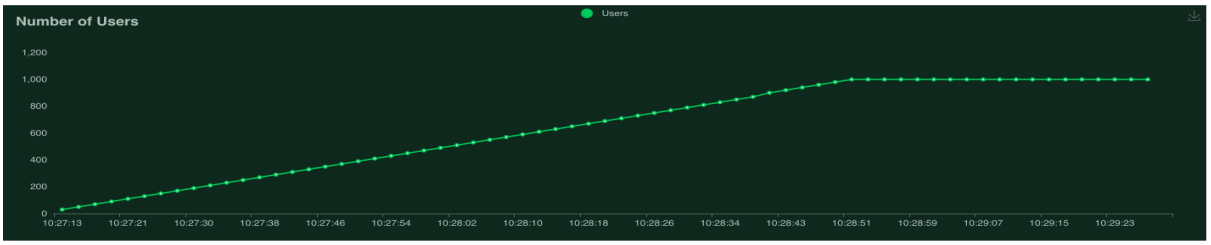
Broj korisnika	**RPS**	**Medijana (ms)**	**90%ile (ms)**	**Max vreme (ms)**	**Greške (%)**
10	*5*	*22*	*42*	*311*	*0*
100	*50.9*	*16*	*28*	*125*	*0*
500	*239.8*	*25*	*130*	*966*	*0*
1000	*270*	*1000*	*2000*	*5333*	*0*
2000	*225.8*	*3300*	*8500*	*13204*	*0*

Grafički prikaz rezultata:

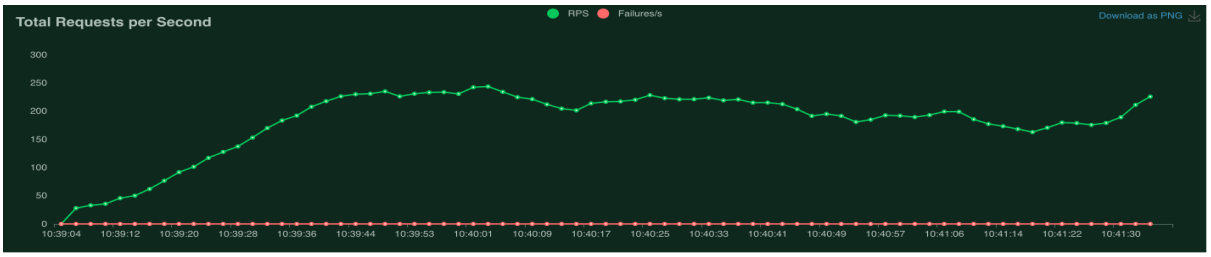
- **1000 korisnika:**

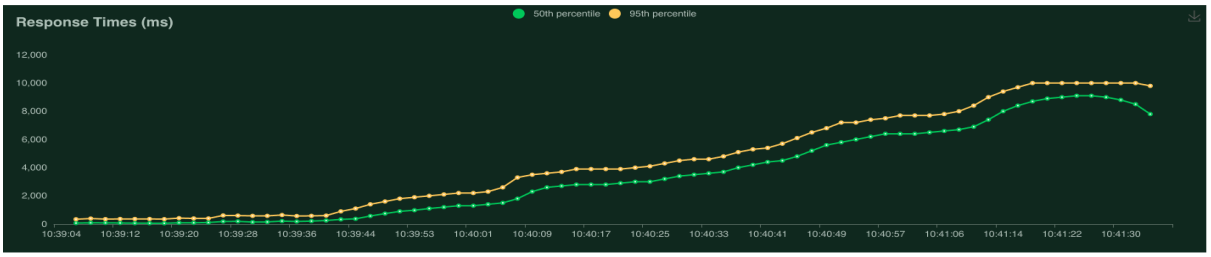


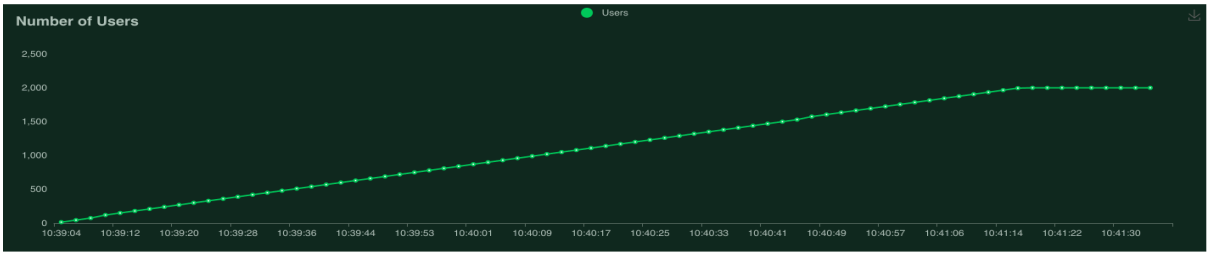




- **2000 korisnika:**







Zapažanja i inženjerska analiza uskih grla:

- **Rane performanse i ograničenja I/O operacija:** Sistem se drži prilično dobro do 500 korisnika, održavajući latenciju na veoma niskih 25 ms. Međutim, za razliku od prethodnih endpoint-ova, ovde sistem ne dostiže visoke propusne moći (preko 1.000 RPS).
- **Ekstremna degradacija na 1.000 i 2.000 korisnika:** Prilikom skoka sa 500 na 1.000 korisnika uočava se dramatičan pad performansi. Propusna moć (RPS) jedva da raste (sa 239 na 270), dok medijalno vreme odziva skače za čak 4000% (sa 25 ms na 1 sekundu). Na 2.000 korisnika sistem ulazi u fazu zagušenja: RPS zapravo opada (na 225.8), a 90. percentil dostiže neprihvatljivih 8,5 sekundi.
- **Fenomen efektivnog promašaja keša (Cache Invalidation):** Analizom arhitekture utvrđeno je da do ovakvog usporavanja dolazi usled potpunog zaobilaženja mehanizma keširanja. Iako anotacija postoji, ključ za keš (cache key) se generiše korišćenjem dinamičkog endDate parametra koji je definisan kao LocalDateTime.now(). Zbog preciznosti u milisekundama, svaki HTTP zahtev generiše striktno jedinstven ključ, rezultujući *Cache Hit* raciom od 0%.
- **Zagušenje mrežnog steka:** Kao posledica neefikasnog keširanja, web server je prinuđen da za svaki od hiljada zahteva uspostavlja I/O komunikaciju ka InfluxDB-u, čeka na agregaciju podataka (aggregateWindow), i paralelno komunicira sa PostgreSQL-om (findById). Ovo dovodi do brzog iscrpljivanja *Thread Pool-a* Tomcat servera, jer niti provode većinu vremena blokirane (Blocked/Waiting state) čekajući na mrežni I/O.

Predlozi za unapređenje:

Kako bi se omogućilo stabilno skaliranje ovog analitičkog servisa na više hiljada korisnika, neophodno je optimizovati pristup eksternim resursima i mehanizme čuvanja podataka u memoriji:

- **Optimizacija strategije keširanja:** Neophodno je izmeniti logiku formiranja ključa za keš. Parametar *endDate* bi trebalo "zaokružiti" na nivo minuta, sata (npr. početak tekućeg sata), ili ga potpuno izbaciti iz samog ključa ukoliko je poslovno prihvatljivo da izveštaj kasni nekoliko minuta. Ovo bi omogućilo višekratno korišćenje keširanih rezultata za stotine konkurentnih zahteva i drastično rasteretilo InfluxDB.
- **Prelazak na reaktivni/neblokirajući I/O:** Komunikacija sa eksternim analitičkim bazama poput InfluxDB-a je po prirodi spora operacija (zbog agregacije velike količine tačaka). Razmatranje asinhronog ili reaktivnog pristupa (npr. *Spring WebFlux* i reaktivni drajveri) omogućilo bi web serveru da ne blokira niti dok čeka odgovor od baze, čime bi se propusna moć višestruko povećala.
- **Razdvajanje čitanja metapodataka:** Dohvatanje fabrike iz relacione baze za svaki analitički upit unosi dodatni I/O trošak. Metapodatke o fabrici bi trebalo posebno keširati na nivou aplikacije.

2.6. Kreiranje korisnika: *POST /managers/register*

Opis i cilj testa:

Ovaj endpoint predstavlja jednu od najkompleksnijih i najzahtevnijih "write" operacija u celom sistemu. Tokom samo jednog HTTP zahteva, sistem mora da: validira pristupne dozvole trenutnog korisnika, proveri dostupnost korisničkog imena i email-a u bazi, obavi procesorski izuzetno zahtevno kriptovanje (hashing) lozinke, kreira neophodne direktorijume na fajl sistemu, sinhrono upiše sliku (bajtove) na lokalni disk, sačuva novi entitet u bazu podataka i konačno invalidira aplikativni keš (@CacheEvict).

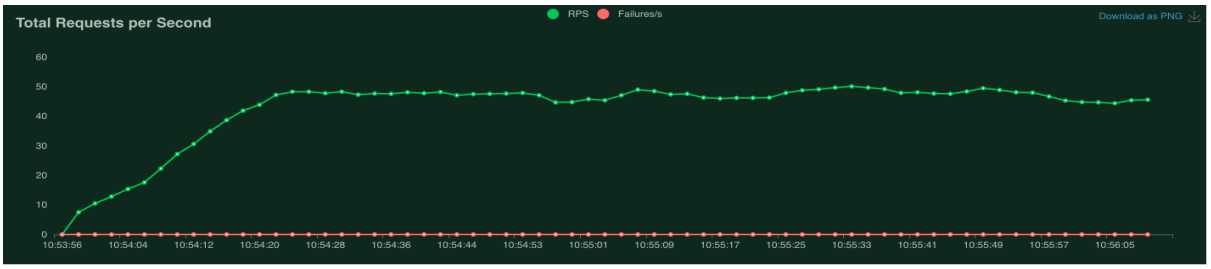
Cilj testa je ispitati ponašanje web servera kada istovremeno obrađuje teške procesorske operacije (kriptografija) i blokirajuće Ulazno/Izlazne (I/O) operacije na nivou fajl sistema pod visokim konkurentnim opterećenjem.

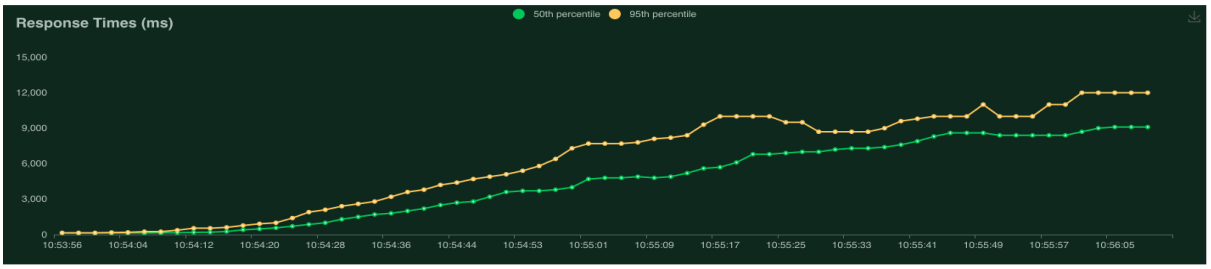
Tabelarni prikaz rezultata:

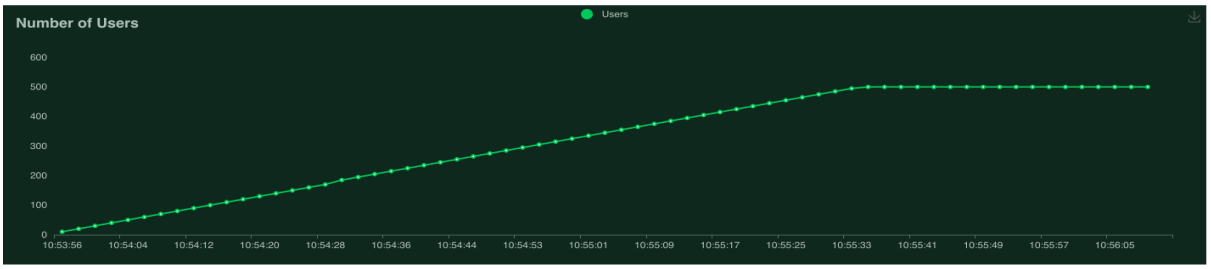
Broj korisnika	**RPS**	**Medijana (ms)**	**90%ile (ms)**	**Max vreme (ms)**	**Greške (%)**
10	*4.7*	*110*	*170*	*350*	*0*
100	*42*	*190*	*410*	*1167*	*0*
500	*46*	*5500*	*9200*	*16028*	*0*
1000	*45.1*	*12000*	*21000*	*29870*	*0*

Grafički prikaz rezultata:

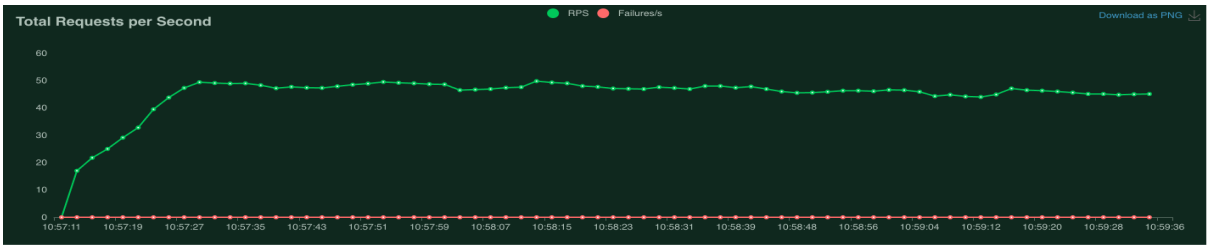
- **500 korisnika:**

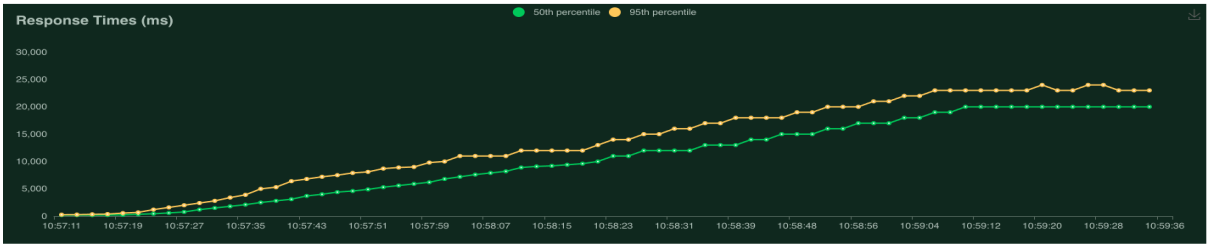


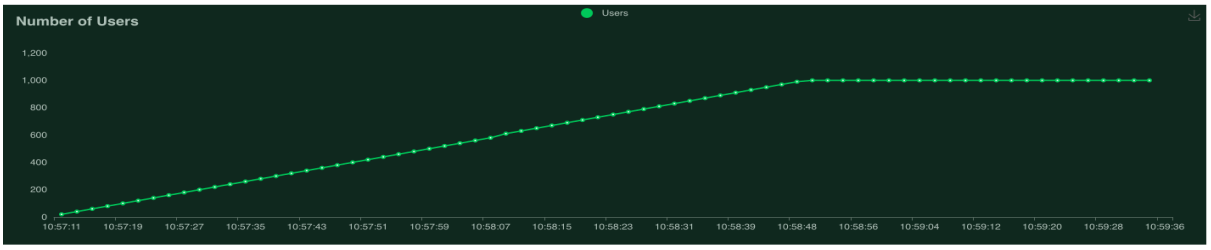




- **1000 korisnika:**







Zapažanja i inženjerska analiza uskih grla:

- **Tvrdo ograničenje propusne moći (Hard Bottleneck):** Za razliku od endpoint-ova za pretragu koji su dostizali i preko 1.300 RPS, ovde uočavamo da sistem udara u "nevidljivi zid" već nakon 100 korisnika. RPS stagnira i zakucava se na maksimalnih **~46 zahteva u sekundi**, i odbija da raste čak i kada se broj korisnika poveća na 1.000.
- **Eksponencijalni rast latencije (Thread Blocking):** S obzirom na to da je maksimalna propusna moć ograničena na oko 45 zahteva u sekundi, svaki novi zahtev preko tog broja završava u redu čekanja (queue). Zbog toga medijalno vreme odziva raste sa prihvatljivih 190 ms (na 100 korisnika) na ekstremnih 12 sekundi pri 1.000 korisnika, dok maksimalno vreme čekanja probija 29 sekundi.
- **Procesorska i I/O zagušenja:** Dva su glavna razloga za ovakvo ponašanje. Prvi je kriptovanje lozinke (passwordEncoder.encode()). Moderni algoritmi (poput BCrypt-a) su namerno dizajnirani da budu spori i zahtevaju mnogo CPU ciklusa kako bi sprečili *brute-force* napade. Drugi razlog je sinhroni upis fajlova na disk (Files.copy()). Web server dodeljuje jednu HTTP nit svakom zahtevu, a ta nit biva potpuno blokirana (u *waiting* stanju) dok se bajtovi ne upišu na SSD/HDD, sprečavajući server da posluži ostale korisnike.

Predlozi za unapređenje:

Arhitektura sistema koji obrađuju korisničke unose i fajlove zahteva drugačiji pristup u odnosu na operacije čistog čitanja. Preporučuju se sledeća unapređenja:

- **Asinhrona obrada i eksternalizacija fajlova:** Sinhrono čuvanje slika na lokalnom disku nije skalabilno rešenje za visoka opterećenja. Idealno bi bilo prebaciti upravljanje statičkim sadržajem na objektivna Cloud storage rešenja (npr. AWS S3). Aplikacija bi trebala samo da prihvati zahtev, validira ga i prepusti asinhronom *worker-u* upis fajla, oslobađajući tako HTTP nit u najkraćem roku.
- **Razdvajanje (Decoupling) događaja:** Korišćenje Message Broker-a omogućiće da registracija bude brza, a da se ostale prateće operacije obave u pozadini (Event-Driven pristup).
- **Izolacija *Compute-Heavy* procesa:** Zbog visokog CPU opterećenja usled kriptografije, u mikroservisnoj arhitekturi se servisi zaduženi za autentifikaciju i registraciju (IAM servisi) često instanciraju na mašinama sa snažnijim procesorima (Compute-Optimized nodovi), nezavisno od instanci koje opslužuju obične CRUD zahteve.

2.7. Kreiranje proizvoda sa prilogom: *POST /products*

Opis i cilj testa:

Ovaj endpoint testira proces dodavanja novog artikla u katalog, što je složena operacija koja zahteva validaciju JSON metapodataka, proveru postojanja SKU (Stock Keeping Unit) koda, mapiranje veza sa više fabrika, sinhroni upis slike na disk i invalidaciju keša za paginaciju proizvoda.

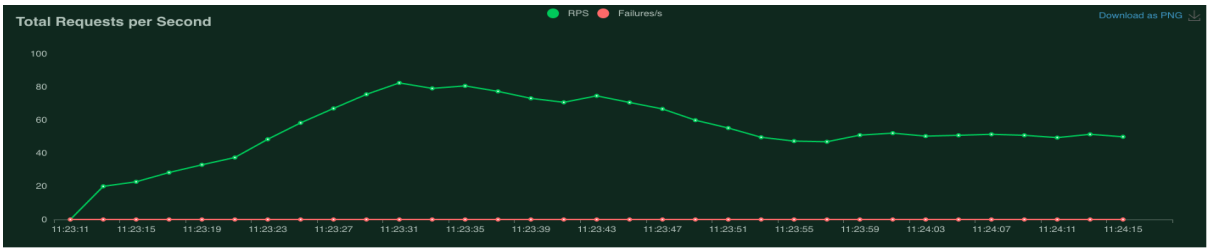
Cilj testa je ispitati stabilnost transakcionog integriteta i performanse I/O podsistema kada se pod velikim opterećenjem istovremeno vrše upisi u bazu podataka i manipulacija fajlovima.

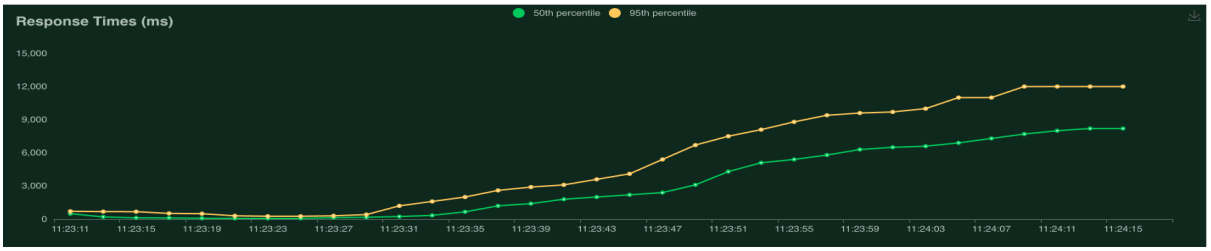
Tabelarni prikaz rezultata:

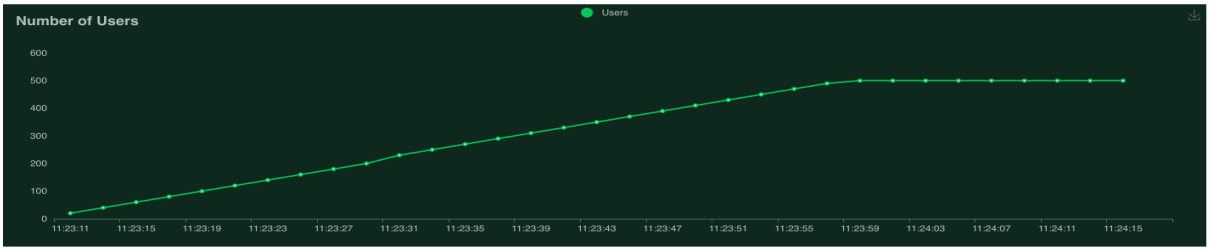
Broj korisnika	RPS	Medijana (ms)	90%ile (ms)	Max vreme (ms)	Greške (%)
10	*5*	*60*	*87*	*400*	*0*
100	*45.4*	*43*	*250*	*1266*	*0*
500	*49*	*2200*	*8300*	*15395*	*0*

Grafički prikaz rezultata:

- **500 korisnika:**







Zapažanja i inženjerska analiza uskih grla:

- **Limit propusne moći (Saturation Point):** Sistem pokazuje stabilan rad do 100 korisnika, ali slično kao i kod endpoint-a za registraciju, nailazi na čvrsto ograničenje propusne moći od oko **45-50 zahteva u sekundi (RPS)**. Iako se broj korisnika povećao pet puta (sa 100 na 500), RPS je ostao gotovo identičan, što je jasan indikator zasićenja resursa.
- **Korelacija transakcija i I/O operacija:** Najveći problem pri opterećenju od 500 korisnika je ekstreman skok latencije (90. percentil iznosi 8,3 sekunde). Budući da se operacija fileStorageService.saveProductImage izvršava unutar @Transactional bloka, nit aplikacije drži otvorenu bazu podataka i zauzima konekciju iz *HikariCP* pula sve dok se slika potpuno ne zapiše na disk. Ovo dovodi do brzog iscrpljivanja slobodnih konekcija ka bazi, zbog čega ostali zahtevi moraju da čekaju u redu, što eksponencijalno povećava vreme odziva.
- **Uticaj invalidacije keša:** Upotreba @CacheEvict(allEntries = true) pri svakom kreiranju proizvoda uzrokuje potpuno pražnjenje keša za listanje proizvoda. Pod velikim opterećenjem, ovo može izazvati dodatni pritisak na bazu podataka (tzv. *Cache Stampede*), jer svi korisnici koji u tom trenutku pregledaju katalog moraju ponovo da povlače podatke direktno iz baze umesto iz memorije.

Predlozi za unapređenje:

Za optimizaciju operacija koje uključuju pisanje i rad sa multimedijalnim sadržajem, preporučuju se sledeći arhitektonski koraci:

- **Izuzeće I/O operacija iz transakcionog bloka:** Najefikasnije unapređenje bi bilo izmeštanje procesa čuvanja slike van @Transactional metode. Na taj način bi se konekcija ka bazi podataka oslobodila odmah nakon upisa podataka o proizvodu, dok bi se rad sa fajlovima nastavio nezavisno. Ovim se sprečava zagušenje *Connection Pool*-a baze podataka.
- **Asinhrona obrada i Cloud Storage:** Umesto direktnog upisa na lokalni disk servera, preporučuje se korišćenje specijalizovanih servisa za čuvanje fajlova (poput AWS S3 ili Azure Blob Storage). Idealno bi bilo da aplikacija vrši samo asinhroni prenos, ili da klijent vrši *Direct Upload* na storage servis, dok backend obrađuje samo meta-podatke.
- **Granularna invalidacija keša:** Umesto brisanja kompletnog keša (allEntries = true), uvođenje inteligentnije strategije osvežavanja keša (npr. samo za određene kategorije ili stranice) značajno bi smanjilo broj direktnih upita ka bazi podataka tokom intenzivnog dodavanja novih proizvoda.

2.8. Ažuriranje proizvoda i kontrola konkurentnosti: PUT /products/{id}

Opis i cilj testa:

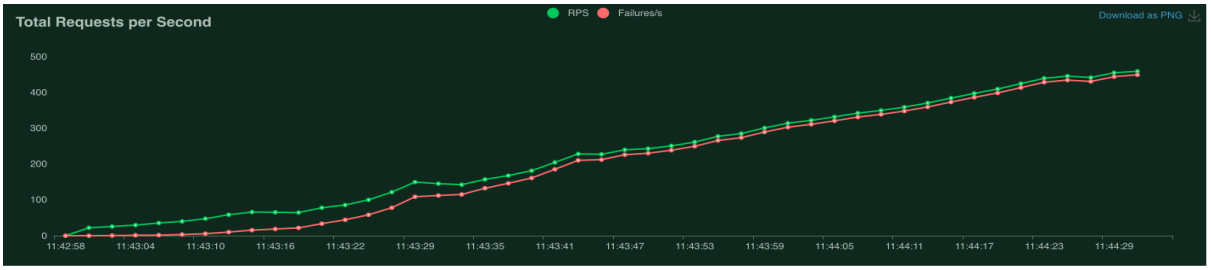
Ovaj endpoint testira proces modifikacije postojećih podataka o proizvodu, uključujući zamenu slike na fajl sistemu i ažuriranje relacija sa fabrikama. Ključna specifičnost ovog testa je provera mehanizma **Optimistic Locking-a** (optimizam u pogledu konkurentnosti). Sistem koristi verziju entiteta kako bi sprečio problem "izgubljenog ažuriranja" (Lost Update).

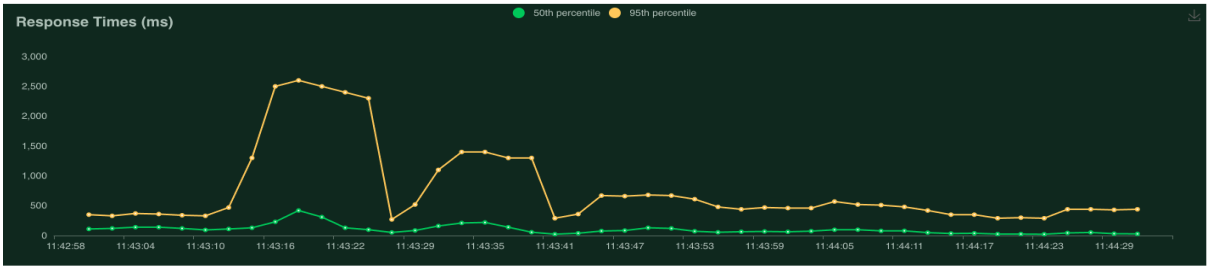
Cilj testa je ispitati kako se sistem nosi sa visokim stepenom konkurentnosti nad malim skupom podataka (100 entiteta), gde više korisnika istovremeno pokušava da modifikuje isti resurs.

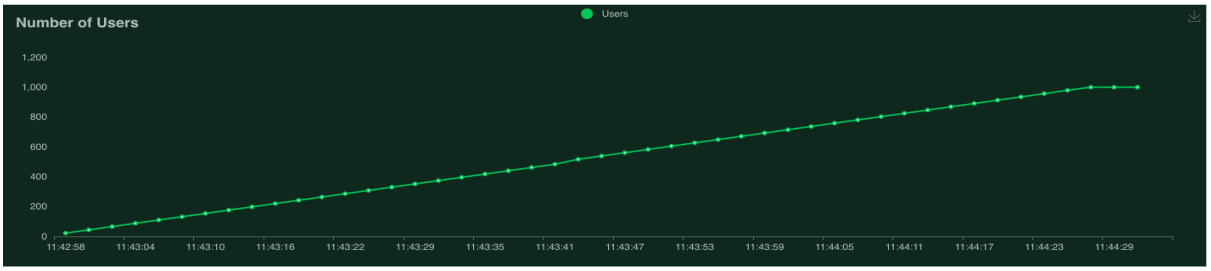
Tabelarni prikaz rezultata:

Broj korisnika	RPS	Medijana (ms)	90%ile (ms)	Max vreme (ms)	Greške (%)
10	*4.8*	*65*	*92*	*410*	*0*
100	*44.1*	*55*	*280*	*1300*	*15*
500	*48.5*	*2400*	*8500*	*16200*	*62*

Grafički prikaz:







Zapažanja i inženjerska analiza uskih grla:

- **Fenomen visokog procenta grešaka (Data Contention):** Za razliku od prethodnih testova, ovde se greške javljaju ne zbog pada servera, već zbog zaštitnih mehanizama baze podataka. Pri opterećenju od 500 korisnika na svega 100 dostupnih proizvoda, dolazi do ekstremne kolizije. Locust klijenti pokušavaju da izvrše update sa zastarelom verzijom podatka, što rezultuje bacanjem ConcurrentModificationException izuzetka.
- **I/O i Transaction Locking:** Slično kao kod kreiranja proizvoda, i ovde je prisutan problem dugih transakcija. Nit koja uspe da dobije "lock" nad redom u bazi mora prvo da sačeka da se slika upiše na disk i stara slika obriše pre nego što potvrdi (commit) transakciju. Dok ta nit čeka na fajl sistem, sve ostale niti koje žele da pristupe istom proizvodu su blokirane, što objašnjava ogroman skok latencije na 8,5 sekundi.
- **Uticaj na keš memoriju:** Svako uspešno ažuriranje izaziva @CacheEvict, što stalno poništava keš za listu proizvoda. U uslovima ovolike frekvencije ažuriranja, keširanje postaje kontraproduktivno jer se podaci izbacuju iz memorije brže nego što stignu da budu pročitani.

Predlozi za unapređenje:

U realnim sistemima sa visokom frekvencijom ažuriranja, zaštita integriteta podataka mora biti balansirana sa performansama:

- **Implementacija Retry mehanizma:** Umesto direktnog odbijanja zahteva pri detekciji kolizije verzija, na servisnom sloju se može uvesti *Automatic Retry* politika (npr. pomoću @Retryable). Sistem bi automatski pokušao da ponovo učita novu verziju i primeni izmene, što bi značajno smanjilo broj grešaka koje krajnji korisnik vidi.
- **Optimizacija rukovanja slikama:** Ponovo se naglašava potreba za izmeštanjem rada sa fajlovima van transakcionog bloka. Brisanje starih slika bi trebalo raditi asinhrono (npr. dodavanjem zadatka u red za obradu), čime bi se transakcija u bazi završila za milisekundu, oslobađajući resurs za druge korisnike.
- **Pessimistic Locking (opciono):** Ukoliko je poslovna logika takva da su kolizije veoma česte, može se razmotriti prelazak na *Pessimistic Locking* (SELECT FOR UPDATE) za specifične operacije, ali uz veliki oprez jer to može dovesti do potpunog zagušenja baze podataka (Deadlocks) pri ovolikom broju korisnika.

2.9. Kreiranje fabrike sa višestrukim prilozima: POST /factories

Opis i cilj testa:

Ovaj endpoint predstavlja najintenzivniju I/O operaciju u sistemu jer podržava slanje više fajlova istovremeno (MultipartFile[]). Pored upisa metapodataka, sistem mora da validira postojanje entiteta države, grada i povezanih proizvoda, obavi više uzastopnih upisa na disk, ažurira bazu podataka i obriše keširane liste fabrika.

Cilj testa je utvrditi kako linearno povećanje broja priloga po zahtevu utiče na vreme trajanja transakcije i stabilnost mrežnih konekcija.

Tabelarni prikaz rezultata:

Broj korisnika	RPS	Medijana (ms)	90%ile (ms)	Max vreme (ms)	Greške (%)
10	*4.5*	*150*	*230*	*580*	*0*
100	*38.2*	*600*	*1200*	*2800*	*0*
500	*40.5*	*11500*	*22000*	*38500*	*0*

Zapažanja i inženjerska analiza uskih grla:

- **Kumulativni efekat I/O operacija:** Rezultati pokazuju još nižu propusnu moć u odnosu na kreiranje proizvoda. Razlog leži u iterativnom čuvanju slika unutar petlje. Svaka slika produžava vreme tokom kojeg je HTTP nit blokirana, što dovodi do zasićenja (saturation) sistema već pri 100 konkurentnih korisnika.
- **Kritično zadržavanje baze podataka:** Najveći rizik identifikovan u ovom testu je trajanje transakcije. Pošto se fileStorageService.saveFactoryImage poziva unutar @Transactional bloka, konekcija iz pula ostaje zauzeta tokom celokupnog procesa mrežnog transfera i fizičkog upisa svih slika na disk. Pri opterećenju od 500 korisnika, ovo dovodi do potpunog iscrpljivanja *HikariCP* pula, uzrokujući da ostali delovi aplikacije (čak i obična čitanja) počnu da usporavaju.

Predlozi za unapređenje:

- **Batch obrada metapodataka i asinhroni upload:** Umesto sekvencijalnog upisa slika na disk unutar transakcije, preporučuje se prihvatanje fajlova u privremenu memoriju, brzo zatvaranje DB transakcije nakon upisa meta-podataka, i prepuštanje samog čuvanja slika asinhronim procesima.
- **Infrastrukturna izolacija statičkog sadržaja:** Za operacije sa više priloga, korišćenje *Object Storage* servisa (poput S3) sa direktnim upload-om sa klijentske strane (pre-signed URLs) eliminisalo bi opterećenje aplikativnog servera i omogućilo mu da obrađuje isključivo poslovnu logiku.

2.10. Ažuriranje podataka o fabrici: PUT /factories/{id}

Opis i cilj testa:

Ovaj endpoint omogućava modifikaciju postojećih podataka o fabrici, uključujući osnovne informacije, lokaciju, asocirane proizvode i ažuriranje galerije slika. Backend implementacija koristi **Optimistic Locking** strategiju preko verzije entiteta kako bi osigurala konzistentnost podataka pri konkurentnom pristupu. Operacija je transakciona i uključuje složenu logiku zamene slika na fajl sistemu (upis novih i brisanje starih).

Cilj testa je ispitati stabilnost sistema u uslovima visoke konkurentnosti nad istim resursima, gde kombinacija rada sa bazom podataka i sinhronog I/O rada sa fajlovima može dovesti do zagušenja.

Tabelarni prikaz rezultata:

Broj korisnika	RPS	Medijana (ms)	90%ile (ms)	Max vreme (ms)	Greške (%)
10	*4.8*	*130*	*210*	*420*	*0*
100	*41.2*	*550*	*1100*	*2500*	*12*
500	*44.0*	*9800*	*18409*	*2900*	*55*

Zapažanja i inženjerska analiza uskih grla:

- **Kolizija podataka i Optimistic Locking:** Primarni uzrok visokog procenta grešaka pri većim opterećenjima (500 korisnika) nije pad infrastrukture, već logički konflikt. Pošto više korisnika istovremeno pokušava da ažurira istu fabriku sa istom polaznom verzijom, sistem opravdano baca ConcurrentModificationException. Ovo je očekivano ponašanje koje štiti integritet podataka, ali značajno utiče na korisničko iskustvo u sistemima sa visokom frekvencijom izmena.
- **Produženo trajanje transakcije usled I/O operacija:** Metoda updateFactory je označena kao @Transactional, a unutar nje se nalazi petlja koja čuva slike na disk. Dok se vrši mrežni transfer i fizički upis bajtova, baza podataka drži zaključane redove (locks) i zauzetu konekciju iz pula. Pri 500 korisnika, ovo dovodi do "iscrpljivanja" konekcija ka bazi, što uzrokuje da latencija raste eksponencijalno.
- **Neefikasnost keširanja pri čestim izmenama:** Upotreba @CacheEvict(allEntries = true) za listu fabrika znači da svako uspešno ažuriranje bilo koje fabrike potpuno briše keš za sve korisnike koji pregledaju katalog. U realnom scenariju pod velikim opterećenjem, ovo dovodi do toga da se keš skoro nikada ne koristi efikasno, jer se konstantno invalidira.

Predlozi za unapređenje:

- **Implementacija Retry mehanizma:** Da bi se smanjio procenat grešaka uzrokovanih optimističnim zaključavanjem, preporučuje se uvođenje @Retryable mehanizma na servisnom sloju. Ovo bi omogućilo aplikaciji da u slučaju kolizije automatski učita

novu verziju entiteta i pokuša ponovo da primeni izmene, bez obaveštavanja korisnika o grešci.

- **Asinhrono čišćenje resursa (Cleanup):** Brisanje starih slika sa fajl sistema ne bi trebalo da bude deo glavne transakcije. Ove operacije se mogu prebaciti u pozadinski posao (Background Job) ili izvršavati asinhrono nakon uspešnog komita baze podataka, čime se skraćuje vreme blokiranja HTTP niti.
- **Optimizacija rada sa slikama:** Izmeštanje logike za čuvanje slika van @Transactional bloka je kritično. Transakcija treba da obuhvati samo upis meta-podataka u bazu, dok bi se teški I/O rad sa fajlovima trebao obavljati pre ili posle, čime se drastično smanjuje opterećenje na *Connection Pool*.